# Discussion Module:

## DSE Analytics, Capacity Planning and Tuning

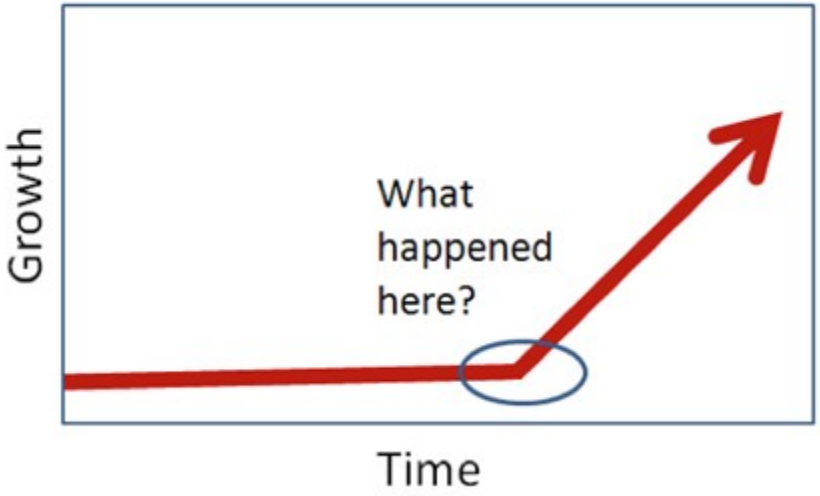
Discussion/Information Only-

Anything less than (n) nodes and (mGB) data can produce spurious results. A capacity planning and tuning lab exists in the advanced class; 14 + nodes, plus driver nodes, production sized data, other.

- Topics related to capacity planning
- Topics related to tuning
- DSE Analytics specific only

# Discussion Lab:

Discuss



# Sizing a (Standard Relational Database):

- INSERT
- UPDATE
- DELETE
- SELECT

- Planned maintenance
  - -- Backups
  - -- Index rebuilds
  - -- ..
- Other overhead
  - -- Maintaining a warm backup site
  - -- ..
- Other-

# Sizing (Analytics):

(Indexes) of activity/jobs-

- Frequency
- Concurrency
- Spread

Other-

- Data feeds (in)
  Data feeds (out)
- ..

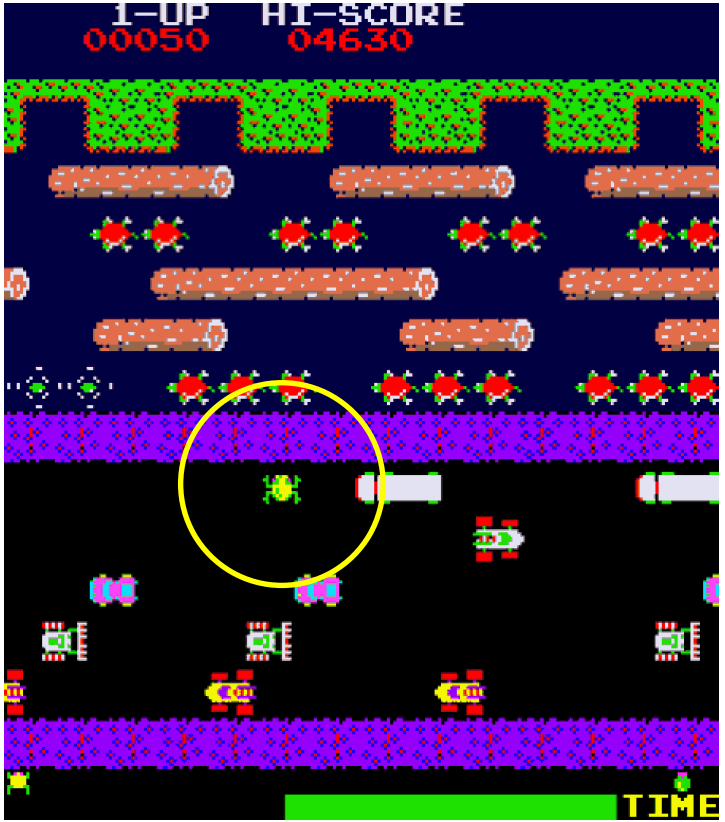
# End of Discussion Lab:

# DSE Analytics: Cap Planning, Tuning-

System tuning

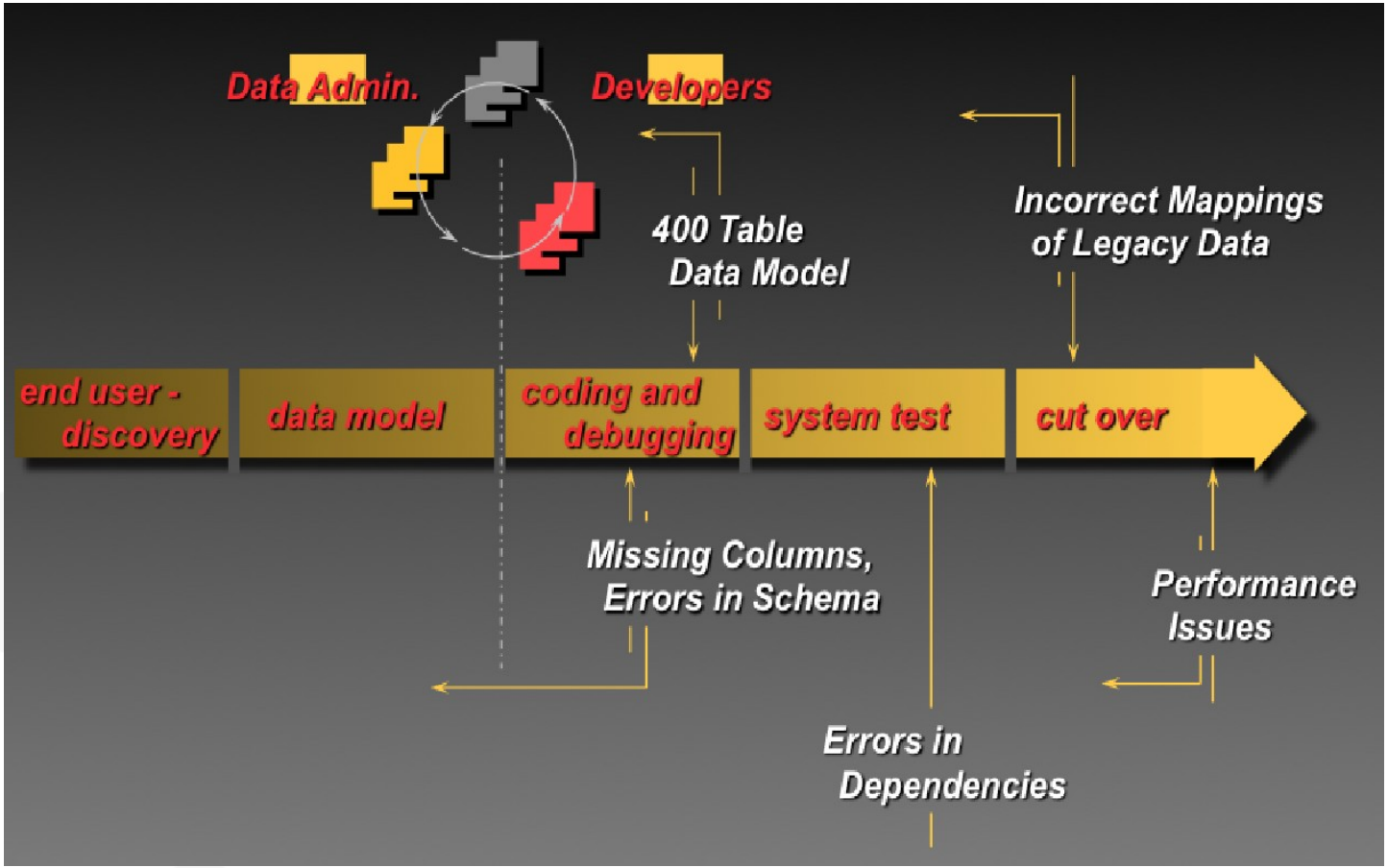
DSE Analytics job tuning

- Prod sized data, repeatable test harness, isolation
- Frequency, concurrency, spread
- One variable; rinse, repeat

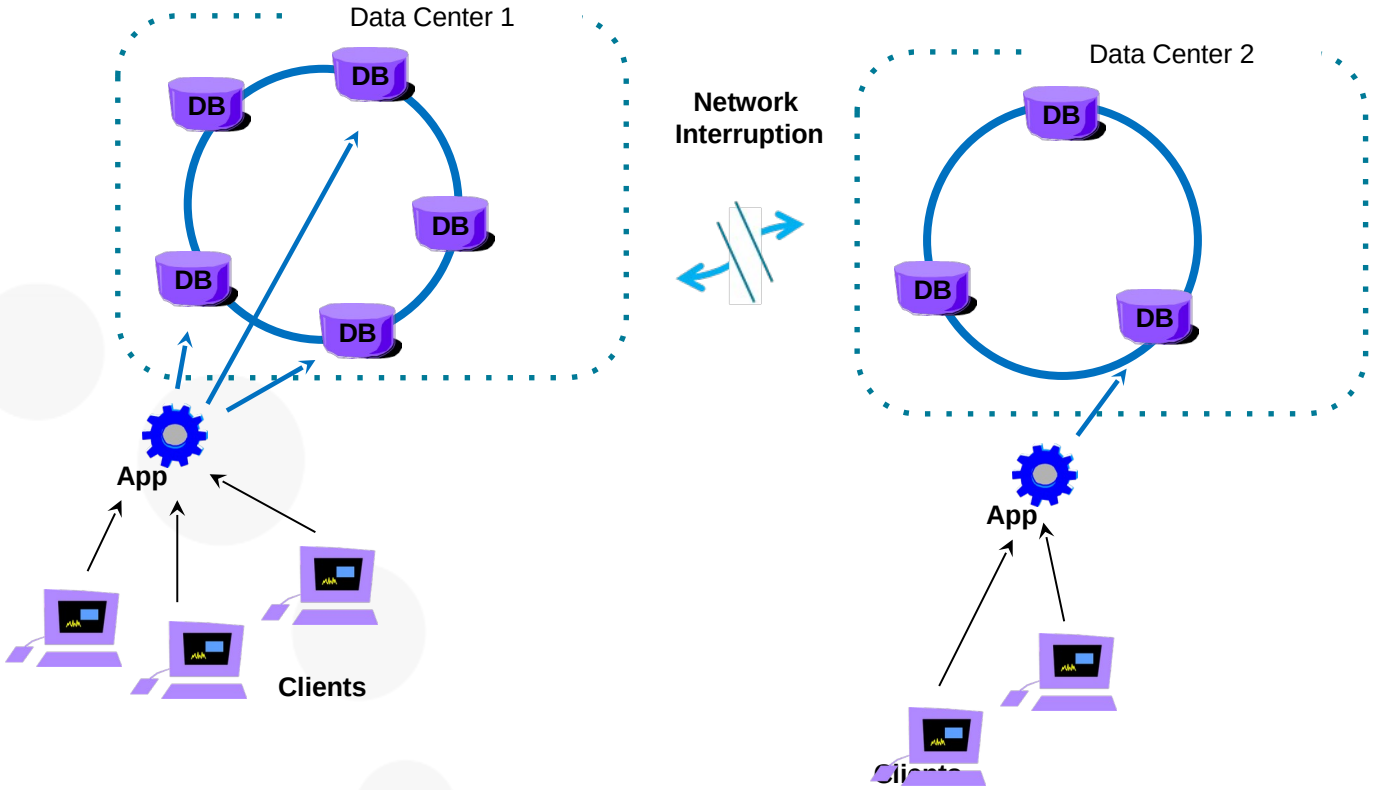



http://www.classicgaming.cc/classics/frogger/about

# Common SDLC, and Issues



# DSE Analytics: DCs, Load isolation



# If you don’t have the Analytics Routine Test Harness-

1

- Developers
  - This is something someone should have created/maintained
  - Reverse engineering is not forecasting, signing your name to something

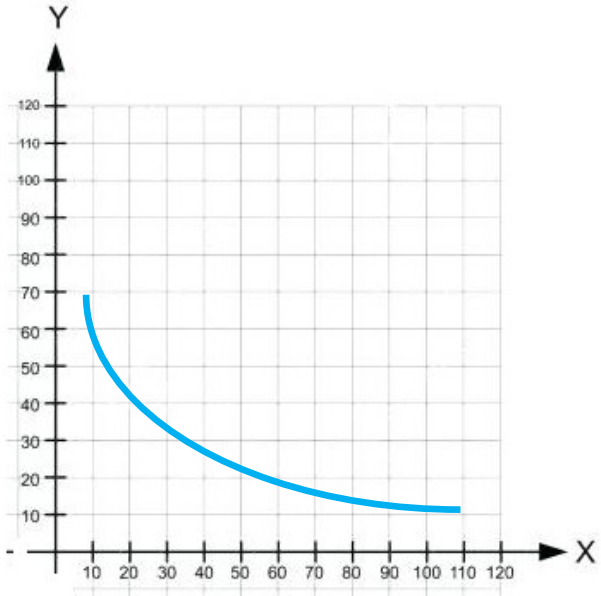
2

- System log
- DSE Analytics History Server
- Application logs
- (Other)

- DSE Audit Secure subsystem
  - https://docs.datastax.com/en/dse/6.0/dse-admin/datastax_enterprise/security/secAuditEnable.html

# Start working the (graph)

Duration




Frequency of Analytics Routine/Job

- Not cost effective to fix every DSE Analytics routine/job; Pick the top n% that you have time for

# DSE Analytics

Configuring, Startup and more

# DSE Analytics: How Spark is started

/etc/default/dse file

- By default setup as part of the package installers
- A copy if you were to manually do it is in the tarball
  - -- <tar extract location>/resources/dse/conf/dse.default
  - -- If creating a custom init script copy to /etc/default or have script point to a custom location
- Allows you to turn on/off on startup
  - -- Spark
  - -- DSEFS is not configured here
- If not using init script need to pass in flags , -k for spark

# DSE Analytics: dse.yaml

Found in either,

- /etc/dse/dse.yaml
- <install_dir>/resources/dse/conf/dse.yaml

- Spark cluster and application statistics being collected
- Initial spark worker resources (as a percentage after C*)
- Configuration change in 6.0.0, much more granular
- Spark security and encryption
- Always on SQL Server
- Spark readiness check
- DSEFS (and its configuration)
- Spark Auditing (more as C* but spark-sql audit shows up)
- Solr query paging (key for SearchAnalytics mixed mode)

# DSE Analytics: dse-spark-env.sh

Found in either,

- /etc/dse/spark/dse-spark-env.sh
- <install_dir>/resources/spark/conf/dse-spark-env.sh

Rarely touched

- Mainly for defaults regarding running Spark with DSE
- Most environment changes are done in spark-env.sh

# DSE Analytics: spark-env.sh

Found in either,

- /etc/dse/spark/spark-env.sh
- <install_dir>/resources/spark/conf/spark-env.sh

Set default CORES and MEMORY across

- Workers
- Executors
- Master
- Driver

This allows to fine tune memory and processors rather than the generic % found in the dse.yaml

# DSE Analytics: spark-defaults.conf

Found in the usual Spark configuration locations

- Pass in default Spark properties
- If using encryption specify settings here
- Can use a different file to set defaults for various apps
  - -- dse spark-submit --properties-file new-properties-file
  - -- But there can be only one, if you have something in the spark-defaults.conf but pass in new file it will ignore the spark-defaults.conf
  - -- Property file can be whitespace or = demarcation of property to value
- Available properties to configure are in, http://spark.apache.org/docs/latest/configuration.html#available-properties
- Defaults, what you want for the majority of applications
  - -- Using secondary properties file can set per app as you pass in the job
  - -- Any property can also be a configured individually within your app

# DSE Analytics: spark-defaults.conf

Security settings

- If Cassandra authorization/authentication turned on
  - -- spark.cassandra.auth.username
  - -- spark.cassandra.auth.password
- If using DSEFS with security
  - -- com.datastax.bdp.fs.client.authentication=basic
  - -- com.datastax.bdp.fs.client.authentication.basic.username
  - -- com.datastax.bdp.fs.client.authentication.basic.password
  - -- Other settings and restrictions apply, see http://docs.datastax.com/en/dse/6.0/dse-admin/datastax_enterprise/analytics/authDsefs.html

# DSE Analytics

Tracking, Auditing

# DSE Analytics: Tracking, Auditing

- User interface including stdout/stderr
- Change the log level of various components of Spark
  -- logback-spark.xml
  -- logback-spark-server.xml
  -- logback-spark-executor.xml
  -- Other
- Audit logging in the dse.yaml
  --audit_logging_options
- Spark cluster info in the dse.yaml
  -- spark_cluster_info_options
- Spark application info in the dse.yaml
  spark_application_info_options

# DSE Analytics

Performance

# DSE Analytics: Where to Start

Issues submitting and running jobs-

- Spark Master UI
- Spark Work UI
- stdout/strderr
- Spark Application UI
- Log files
  - -- system.log
  - -- /var/log/spark … (by default)
  - -- master.log
  - -- worker.log
  - -- (application logs)
- OpsCenter for overall cluster health
- Submit job using the –verbose/-v flag for more output

# DSE Analytics: Spark Issues

Problems with Memory-

- OOM
- More helps speed things
- Maybe need more systems in the cluster

Problems with the cores-

- Jobs can’t start
- Jobs waiting to start
- Jobs take a long time to run
- Maybe need more systems in the cluster

Problems with code-

- Classpath issues
- Inefficient code
- Doing a lot of data shuffling

# DSE Analytics: Memory Issues

- Large amount of data on disk (rdd directory) when you were not expecting
  -- Spark tries to do everything in memory, then will spill to disk
  -- If expected everything to fit into memory, why is it spilling
  -- Look at the application UI and the storage details
  -- Look at dag schedule, etc
- Just need more memory
  -- Add nodes
  -- Adjust memory settings for Spark, don't starve DSE Core JVM
  -- How you are programming
    .. Collecting and sending to driver, and back out again excessively?
    .. Filter early, often
    .. Spark will optimize job, but can only do so much

# DSE Analytics: OOM Errors-

First question is where-

- DSE
- Master
- Executor (should not happen)
- Driver/Application

Depending on where, adjustments you can make

- Adjust memory for Spark, don’t starve DSE
- Set more appropriate settings for driver, and executor in your Spark configuration
  - -- Default if it is for all jobs
  - -- Specific job as only if ..
  - -- Driver memory settings for different jobs

# DSE Analytics: Cores

Jobs in waiting state, resource are not available

- Maybe this is ok,  lot of jobs queued and you just want to wait
- Architectural decision; set resource to one job to get done faster, or partial resources to jobs so multiple can run at the same time
- Many executors with smaller memory vs few executors with lots of memory each
- Turning on resource sharing may be an appropriate action
- Could have everything fully utilized because of a lot of open streams jobs that never close
- Could be you are over allocating cores to a job, use your config to adjust

Jobs that are taking a long time to complete

- Find where the bottleneck is I.e you are pulling data from an external system and that is as fast as it can send
- Code Optimization as the solution? Number one thing to fix

# DSE Analytics: Code

- Unnecessarily shuffling data
- Cache appropriately at the right level
- Filter early and often and collect late
- Manipulate/lift only the data needed
- Pulling data from external systems, so one worker is not a bottleneck; I.e. from RDBMS not using a strategy that allows partitioning. (One worker has to pull all data, then transfer it to other workers, so they can then manipulate the data.)
- Coding practices and algorithms, expert/peer code reviews
- Performance test with real data loads, no surprise when going into production

# DSE Analytics: Code

- Writing to DSE Core, code to use single partition batches; don’t have to wait on other mutations
- Utilize fewer coordinator nodes

- Upgrade from older DSE/DSE-Analytics, check release notes

- (Recompile code as APIs may not be backwards compatible)

# DSE Analytics: Miscellaneous

Classpath issues

- Submitting a fat jar that should have all classes; overhead but puts everything on the path
- Local directory for dependent jars passing in the -jars flag
  -- jar right version on all machines in the cluster
  -- Use a shared file system, DSEFS
  -- Submit in cluster mode, jars passed by driver

Max result size exceeded

- spark.driver.maxResultSize is 1GB by default
  http://spark.apache.org/docs/2.0.2/configuration.html
  May need to set higher

# DSE Analytics: Lastly-

If you can’t find the issue using the UI, logs, etc. then it may be time to dive deeper

- Get a thread dump
- Get a heap dump

Understanding code and what it is suppose to be doing is a must; object that is 6GB in size may be expected

Useful tools

- Memory analyzer for heap dumps http://www.eclipse.org/mat/
- FastThreads.io for thread dumps http://fastthread.io/
- Samurai http://samuraism.jp/samurai/en/index.html

# End of Module:

# DSE Analytics: Enabling Logging

- Find/review the following files
  - -- audit.log
  - -- master.log, worker.log
  - -- system.log
  - -- stderr, stdout
- Create a new table in DSE Core
- Turn on
  - -- Auditing query level
  - -- Trace in logback files
  - -- spark_cluster_info_options
  - -- spark_application_info_options
- Restart system
- Load data into the new table from Spark REPL
  - val data = spark.read.format ...
- Review all the files again